



Lab Pratique : Chargement de données avec Pandas

Objectifs

Après avoir complété ce lab, vous serez capable de :

- Utiliser Pandas pour accéder aux données et les visualiser

À propos du Dataset

Le tableau contient une ligne pour chaque produit et plusieurs colonnes.

- **OrderID** : Un identifiant unique pour chaque commande
- **Product** : Le nom du produit acheté
- **Category** : La catégorie à laquelle appartient le produit (ex: Electronics, Furniture, Stationery)
- **Quantity** : Le nombre d'unités achetées pour ce produit
- **Price** : Le prix unitaire du produit
- **Total** : Le coût total pour le produit (calculé comme $\text{Quantity} \times \text{Price}$)
- **OrderDate** : La date à laquelle la commande a été passée
- **CustomerCity** : La ville où réside le client

Vous pouvez voir le dataset ici :

OrderID	Product	Category	Quantity	Price	Total	OrderDate	CustomerCity
1	Laptop	Electronics	2	800	1600	2022-01-10	Nice
2	Smartphone	Electronics	3	600	1800	2022-02-15	Lyon
3	Desk Chair	Furniture	5	150	750	2022-03-12	Cannes
4	Notebook	Stationery	10	2	20	2022-04-05	Hammamet

5	Monitor	Electronics	1	300	300	2022-05-21	M
---	---------	-------------	---	-----	-----	------------	---

Introduction à Pandas

```
In [ ]: # Dépendances nécessaires pour installer le fichier

%pip install xlrd openpyxl
```

```
In [ ]: # Importation de la bibliothèque requise

import pandas as pd
```

```
In [ ]: filename = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/L
df = pd.read_csv(filename)
```

Nous pouvons utiliser la méthode `head()` pour examiner les cinq premières lignes d'un dataframe :

```
In [4]: # Afficher les cinq premières lignes du dataframe

df.head()
```

```
Out[4]:
```

	OrderID	Product	Category	Quantity	Price	Total	OrderDate	CustomerCity
0	1	Laptop	Electronics	2	800	1600	2022-01-10	New York
1	2	Smartphone	Electronics	3	600	1800	2022-02-15	Los Angeles
2	3	Desk Chair	Furniture	5	150	750	2022-03-12	Chicago
3	4	Notebook	Stationery	10	2	20	2022-04-05	Houston
4	5	Monitor	Electronics	1	300	300	2022-05-21	Miami

Nous utilisons le chemin du fichier Excel et la fonction `read_excel` . Le résultat est un dataframe comme précédemment :

```
In [ ]: xlsx_path = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/
df = pd.read_excel(xlsx_path)
df.head()
```

Nous pouvons accéder à la colonne **Quantity** et l'assigner à un nouveau dataframe **x** :

```
In [ ]: # Accès à la colonne Quantity
```

```
x = df[['Quantity']]
x
```

Le processus est illustré dans la figure suivante :

x = df [['Quantity']]

OrderID	Product	Category	Quantity	Price	Total	OrderDate	CustomerCity
0	1 Laptop	Electronics	2	800	1600	2022-01-10	New York
1	2 Smartphone	Electronics	3	600	1800	2022-02-15	Los Angeles
2	3 Desk Chair	Furniture	5	150	750	2022-03-12	Chicago
3	4 Notebook	Stationery	10	2	20	2022-04-05	Houston
4	5 Monitor	Electronics	1	300	300	2022-05-21	Miami

➔

	Quantity
0	2
1	3
2	5
3	10
4	1

Visualisation et Accès aux Données

Vous pouvez également obtenir une colonne sous forme de **series**. Vous pouvez considérer une series Pandas comme un dataframe unidimensionnel (1-D). Utilisez simplement un seul crochet :

```
In [ ]: # Obtenir la colonne sous forme de series
```

```
x = df['Product']
x
```

Vous pouvez également obtenir une colonne sous forme de dataframe. Par exemple, nous pouvons assigner la colonne **Quantity** :

```
In [ ]: # Obtenir la colonne sous forme de dataframe
```

```
x = df[['Quantity']]
type(x)
```

Vous pouvez faire la même chose pour plusieurs colonnes ; il suffit de mettre le nom du dataframe, dans ce cas `df`, et les noms des multiples en-têtes de colonnes enfermés dans des doubles crochets. Le résultat est un nouveau dataframe composé des colonnes spécifiées :

```
In [ ]: # Accès à plusieurs colonnes
```

```
y = df[['Product', 'Category', 'Quantity']]
y
```

Le processus est illustré dans la figure suivante :



Une façon d'accéder à des éléments uniques est la méthode `iloc`, où vous pouvez accéder à la 1ère ligne et à la 1ère colonne comme suit :

```
In [ ]: # Accéder à la valeur sur la première ligne et la première colonne
df.iloc[0, 0]
```

Vous pouvez accéder à la 2ème ligne et à la 1ère colonne comme suit :

```
In [ ]: # Accéder à la valeur sur la deuxième ligne et la première colonne
df.iloc[1,0]
```

Vous pouvez accéder à la 1ère ligne et à la 3ème colonne comme suit :

```
In [ ]: # Accéder à la valeur sur la première ligne et la troisième colonne
df.iloc[0,2]
```

```
In [ ]: # Accéder à la valeur sur la deuxième ligne et la troisième colonne
df.iloc[1,2]
```

```
In [ ]:
```

Ceci est illustré dans l'image suivante :

df.iloc[0, 0]: '1.0'**df.iloc[1, 0]: ' 2.0'****df.iloc[0, 2]: ' Electronics'****df.iloc[2, 2]: 'Furniture'**

	OrderID	Product	Category	Quantity	Price	Total	OrderDate	CustomerCity
0	1	Laptop	Electronics	2	800	1600	2022-01-10	New York
1	2	Smartphone	Electronics	3	600	1800	2022-02-15	Los Angeles
2	3	Desk Chair	Furniture	5	150	750	2022-03-12	Chicago
3	4	Notebook	Stationery	10	2	20	2022-04-05	Houston
4	5	Monitor	Electronics	1	300	300	2022-05-21	Miami

Vous pouvez également accéder à la colonne en utilisant le nom, les exemples suivants sont identiques à ceux ci-dessus :

```
In [ ]: # Accéder à la colonne en utilisant le nom
```

```
df.loc[0, 'Product']
```

```
In [ ]: # Accéder à la colonne en utilisant le nom
```

```
df.loc[1, 'Product']
```

```
In [ ]: # Accéder à la colonne en utilisant le nom
```

```
df.loc[1, 'CustomerCity']
```

```
In [ ]: # Accéder à la colonne en utilisant le nom
```

```
df.loc[1, 'Total']
```

df.loc[0, Product]: ' Laptop'**df.loc[1, CustomerCity]: ' Los Angeles'****df.loc[1, Product]: 'Smartphone'****df.loc[1, Total]: ' 1800.0'**

	OrderID	Product	Category	Quantity	Price	Total	OrderDate	CustomerCity
0	1	Laptop	Electronics	2	800	1600	2022-01-10	New York
1	2	Smartphone	Electronics	3	600	1800	2022-02-15	Los Angeles
2	3	Desk Chair	Furniture	5	150	750	2022-03-12	Chicago
3	4	Notebook	Stationery	10	2	20	2022-04-05	Houston
4	5	Monitor	Electronics	1	300	300	2022-05-21	Miami

Vous pouvez effectuer du **slicing** (découpage) en utilisant à la fois l'index et le nom de la colonne :

```
In [ ]: # Slicing du dataframe

df.iloc[0:2, 0:3]
```

z = df.iloc[0:2, 0:3]

z

	OrderID	Product	Category	Quantity	Price	Total	OrderDate	CustomerCity
0	1	Laptop	Electronics	2	800	1600	2022-01-10	New York
1	2	Smartphone	Electronics	3	600	1800	2022-02-15	Los Angeles
2	3	Desk Chair	Furniture	5	150	750	2022-03-12	Chicago
3	4	Notebook	Stationery	10	2	20	2022-04-05	Houston
4	5	Monitor	Electronics	1	300	300	2022-05-21	Miami



	OrderID	Product	Category
0	1	Laptop	Electronics
1	2	Smartphone	Electronics

```
In [ ]: # Slicing du dataframe en utilisant les noms

df.loc[0:2, 'OrderID':'Category']
```

z = df.loc[0:2, 'OrderID':'Category']

z

	OrderID	Product	Category	Quantity	Price	Total	OrderDate	CustomerCity
0	1	Laptop	Electronics	2	800	1600	2022-01-10	New York
1	2	Smartphone	Electronics	3	600	1800	2022-02-15	Los Angeles
2	3	Desk Chair	Furniture	5	150	750	2022-03-12	Chicago
3	4	Notebook	Stationery	10	2	20	2022-04-05	Houston
4	5	Monitor	Electronics	1	300	300	2022-05-21	Miami



	OrderID	Product	Category
0	1	Laptop	Electronics
1	2	Smartphone	Electronics
2	3	Desk Chair	Furniture

1. Groupby (Regroupement)

La fonction `groupby` permet de diviser les données en groupes selon certains critères, d'appliquer une fonction (somme, moyenne, compte) et de combiner les résultats. C'est l'équivalent du `GROUP BY` en SQL.

```
In [ ]: # Exemple : Calculer Le total des ventes par Catégorie
# Nous regroupons par 'Category' et calculons la somme de 'Total'

ventes_par_categorie = df.groupby('Category')['Total'].sum().reset_index()
ventes_par_categorie
```

2. Pivot Tables (Tables de Pivot)

Une **Pivot Table** est une version multidimensionnelle de `groupby`. Elle permet de réorganiser les données pour voir les relations entre deux variables.

```
In [ ]: # Exemple : Créer une table pivot montrant la quantité vendue par Ville et par P
pivot_exemple = df.pivot_table(index='CustomerCity', columns='Product', values='
pivot_exemple
```

3. Jointures : Merge et Concat

- **concat** : Empile des dataframes (verticalement ou horizontalement).
- **merge** : Combine des dataframes sur la base de colonnes communes (Inner Join, Left Join, etc.).

```
In [ ]: # Création d'un second dataframe factice pour la démonstration
donnees_villes = pd.DataFrame({
    'CustomerCity': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Miami'],
    'Region': ['East', 'West', 'Midwest', 'South', 'South']
})

# Utilisation de merge pour ajouter la région à notre dataframe principal
df_enrichi = pd.merge(df, donnees_villes, on='CustomerCity', how='left')
df_enrichi.head()
```

Quiz sur les DataFrame

Utilisez une variable `q` pour stocker la colonne **Price** sous forme de dataframe

```
In [ ]: # Écrivez votre code ci-dessous et appuyez sur Shift+Enter pour exécuter
```

► Cliquez ici pour la solution

Assignez la variable `q` au dataframe composé des colonnes **Product** et **Category** :

```
In [ ]: # Écrivez votre code ci-dessous et appuyez sur Shift+Enter pour exécuter
```

► Cliquez ici pour la solution

Accédez à la 2ème ligne et à la 3ème colonne de `df` :

```
In [ ]: # Écrivez votre code ci-dessous et appuyez sur Shift+Enter pour exécuter
```

► Cliquez ici pour la solution

Utilisez la liste suivante pour convertir l'index du dataframe `df` en caractères et assignez-le à `df_new` ; trouvez l'élément correspondant à l'index de ligne `a` et à la colonne `'CustomerCity'` . Ensuite, sélectionnez les lignes de `a` à `d` pour la colonne `'CustomerCity'`

```
In [ ]: new_index=['a','b','c','d','e']
```

► Cliquez ici pour la solution

Calculez la **moyenne du prix** (`Price`) pour chaque **Catégorie** et stockez-le dans une variable `moyenne_prix` .

```
In [ ]: # Votre code ici
```

► Cliquez ici pour la solution

Créez une table de pivot qui affiche le **Total** cumulé (somme) avec les **Catégories** en index et les **Villes** en colonnes.

```
In [ ]: # Votre code ici
```

► Cliquez ici pour la solution

Créez un nouveau dataframe `df_sup` avec une seule ligne (ajoutez une commande factice) et utilisez `pd.concat` pour l'ajouter à `df` .

```
In [ ]: # Votre code ici
```